
Entwicklung einer Android-App zur Akkuvorhersage mit TinyML

Portfolio-Dokumentation

Vertiefung I: Mobile und Ubiquitäre Anwendungen

Vorgelegt von: Keno Schürger
Matrikelnummer: 5023033
Studiengang: Wirtschaftsinformatik (BWI)
Dozenten: Prof. Dr. Karsten Huffstadt,
Prof. Dr. Isabel John
Semester: Sommersemester 2026
Bearbeitungszeit: 25. März – 11. Mai 2026

Würzburg, 12. Mai 2026

Inhaltsverzeichnis

Abbildungsverzeichnis	II
Tabellenverzeichnis	II
0 Ausgangssituation und Zielsetzung	1
0.1 Problemstellung	1
0.2 Zielsetzung	1
0.3 Rahmenbedingungen	2
0.4 Eckdaten des Projekts	2
1 Architekturmodell	3
1.1 Gesamtarchitektur im Überblick	3
1.2 Android-App im Detail	5
1.3 Python-Pipeline im Detail	5
2 Datenmodell	6
2.1 CSV-Schema	6
2.2 Warum genau diese zehn Features?	6
2.3 Speicherung und Export	7
2.4 Datenschutz	7
2.5 Die App in Aktion	8
3 Programmierung	10
3.1 Datensammlung im Service	10
3.2 Auslesen der 10 Features	11
3.3 On-Device-Inferenz mit TFLite	12
3.4 Robustheit – die App muss überleben	13
3.5 Schema-Drift im Logger	14
3.6 Python-Pipeline – Beispiel Segment-Erkennung	15
3.7 Trainings-Verlauf	15
3.8 Code-Umfang und Eigenleistung	16
3.9 Reproduzierbarkeit der Pipeline	17
4 Kurzreferat (Folien)	19
5 Abschließender Erfahrungsbericht	20
5.1 Wöchentliche Einträge	20
5.2 Gesamt-Aufwand	21
5.3 KI-Nutzung	22
5.4 Abschluss-Reflexion	23

Abbildungsverzeichnis

1	Vereinfachter Datenfluss zwischen den Systemkomponenten	3
2	Detaillierte Gesamtarchitektur des Battery-Predictor-Systems	4
3	Main Activity der Battery Data Collector App auf dem Xiaomi 2107113SG (Redmi Note 10 Pro). Links: erste Sekunden nach Service-Start. Rechts: dasselbe Gerät eine Minute später mit fertiger On-Device-Vorhersage. . . .	9
4	Trainings- und Validation-MAE des Conv1D-Modells. Auf dem Multi-Device-Datensatz erreicht das Modell nach 21 Epochen ein Validation-MAE von ca. 5 h. Da EarlyStopping mit <code>restore_best_weights=True</code> konfiguriert ist, wird das beste Modell zurückgegeben.	16

Tabellenverzeichnis

1	Projekt auf einen Blick	2
2	Module der Android-App	5
3	Felder eines CSV-Eintrags	6
4	Im Manifest deklarierte Permissions	8
5	Code-Umfang nach Bereich	16
6	Zeitaufwand nach Woche	21

Quellcode-Verzeichnis

1	Mess-Schleife im DataCollectorService	10
2	CPU-Last als Frequenz-Proxy	11
3	App-Kategorisierung (Auszug)	11
4	Hotspot-Status via Reflection	12
5	Inferenz mit TFLite	12
6	Auto-Restart bei onTaskRemoved	14
7	Schema-Drift im Logger abfangen	14
8	Discharge-Segmente finden	15
9	Pipeline reproduzierbar starten	17

0 Ausgangssituation und Zielsetzung

0.1 Problemstellung

Smartphone-Akkuanzeigen funktionieren auf den meisten Geräten seit Android 5 (2014) nach einem einfachen Prinzip: man rechnet den durchschnittlichen Verbrauch der letzten Stunden linear in die Zukunft. Wer schon einmal mit 30 % Akku „noch 4 Stunden“ im Display hatte und dann nach einem YouTube-Video plötzlich bei 15 % mit „noch 1 Stunde“ stand, kennt das Problem.

Mit Android 12 hat Google im Oktober 2021 ein eigenes ML-basiertes Modell ins System eingebaut (`PowerManager.getBatteryDischargePrediction()`) [?]. Auf Pixel-Geräten nutzt der Settings-Screen diesen Wert; auf Geräten anderer Hersteller (Xiaomi, Samsung) wird er meist gar nicht angezeigt, obwohl die API verfügbar ist.

0.2 Zielsetzung

Ziel ist eine Android-App, die

- alle 30 Sekunden 10 Sensordaten-Features sammelt (Akku, Helligkeit, App-Kategorie, CPU, Temperatur usw.),
- ein eigenes TinyML-Modell on-device einsetzt,
- parallel die Google-API liest und vergleicht,
- alles in eine CSV schreibt, die am PC ausgewertet wird.

Funktional sollte am Ende stehen:

1. Eine Android-App, die im Hintergrund Daten sammelt und überlebt (Foreground-Service, Auto-Restart nach Boot oder Wegwischen).
2. Ein quantisiertes TFLite-Modell (< 15 KB), das in der App on-device läuft und eine Vorhersage in der Notification anzeigt.
3. Eine Trainings-Pipeline am PC, die die exportierte CSV verarbeitet, das Modell trainiert und es zurück in die App deployt.
4. Eine Auswertung, die zeigt, wie gut das eigene Modell gegen die Google-API und einfache Baselines steht.

Dieses Portfolio dokumentiert den *Bau-Aufwand*, die Architektur und die Programmier-Eigenleistung. Die *wissenschaftliche* Auswertung des Multi-Device-Vergleichs ist Gegenstand der separaten Paper-Einreichung (`paper/main.pdf` im selben Repo) und nicht Inhalt dieses Dokuments.

0.3 Rahmenbedingungen

- **Eigenes Gerät:** Xiaomi 2107113SG (Redmi Note 10 Pro), Android 12, 8 GB RAM.
- **Zusätzliche Test-Geräte** (Familie): Google Pixel 7 Pro, Pixel 8 Pro, Pixel 9 Pro XL.
- **Entwicklungsumgebung:** Android Studio (Kotlin), Python 3.13 mit TensorFlow 2.21 am Windows-Notebook.
- **Bearbeitungszeitraum:** 25. März bis 11. Mai 2026 (sieben Wochen).

0.4 Eckdaten des Projekts

Das Projekt umfasst drei Schichten – die Android-App selbst, eine Python-Pipeline am PC für Training und Auswertung, sowie das zurück-deployte TFLite-Modell für die On-Device-Vorhersage. Die folgenden Kennzahlen geben einen ersten Eindruck vom Umfang; Details zu jeder Schicht finden sich in den entsprechenden Hauptkapiteln.

Tabelle 1: Projekt auf einen Blick

Kennzahl	Wert
Bearbeitungszeit	7 Wochen, ≈ 120 h Eigenleistung
Code-Umfang gesamt	≈ 4.350 Zeilen (Kotlin + Python)
Android-App	6 Kotlin-Module (Activity, Service, Logger, Predictor, Receiver, DataPoint)
Python-Pipeline	25 Module in zwei Schichten (<code>methods/</code> , <code>evaluation/</code>)
Test-Geräte	4 (Xiaomi 2107113SG + Pixel 7 / 8 / 9 Pro)
Mitwirkende	3 Familienmitglieder für APK-Rollout und Pixel-Datensammlung
Feld-Sammlung	45 Tage Dauerbetrieb, 180 Service-Sessions
Datensatz	66.001 Messpunkte $\times$ 10 Features
TFLite-Modell	14.4 KB INT8-quantisiert, $\sim 5 \mu s$ pro Inferenz
App-Vergleich in der UI	3 Werte (TinyML on-device, Google-API, Linear-Baseline)

1 Architekturmodell

1.1 Gesamtarchitektur im Überblick

Das System gliedert sich in zwei klar getrennte Hälften, die durch CSV-Dateien und das `.tflite`-Modell als Schnittstellen kommunizieren:

1. **Datenerhebung (Android-App)** – alle 30 Sekunden ein Datenpunkt mit 10 Features, gespeichert in CSV.
2. **Modelltraining (Python-Pipeline)** – liest die konsolidierte CSV, trainiert Conv1D, konvertiert nach TFLite.
3. **On-Device-Inferenz (Android-App)** – das TFLite-Modell läuft direkt im Phone und liefert die Vorhersage in der Notification.

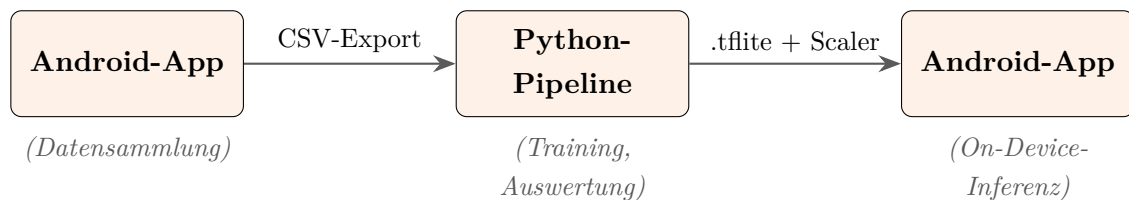


Abbildung 1: Vereinfachter Datenfluss zwischen den Systemkomponenten

Die vollständige Detail-Architektur mit allen Unterkomponenten, internen Verbindungen und Schnittstellen zeigt Abbildung 2 auf der folgenden Seite.

Gesamtarchitektur: Akkuvorhersage mit TinyML

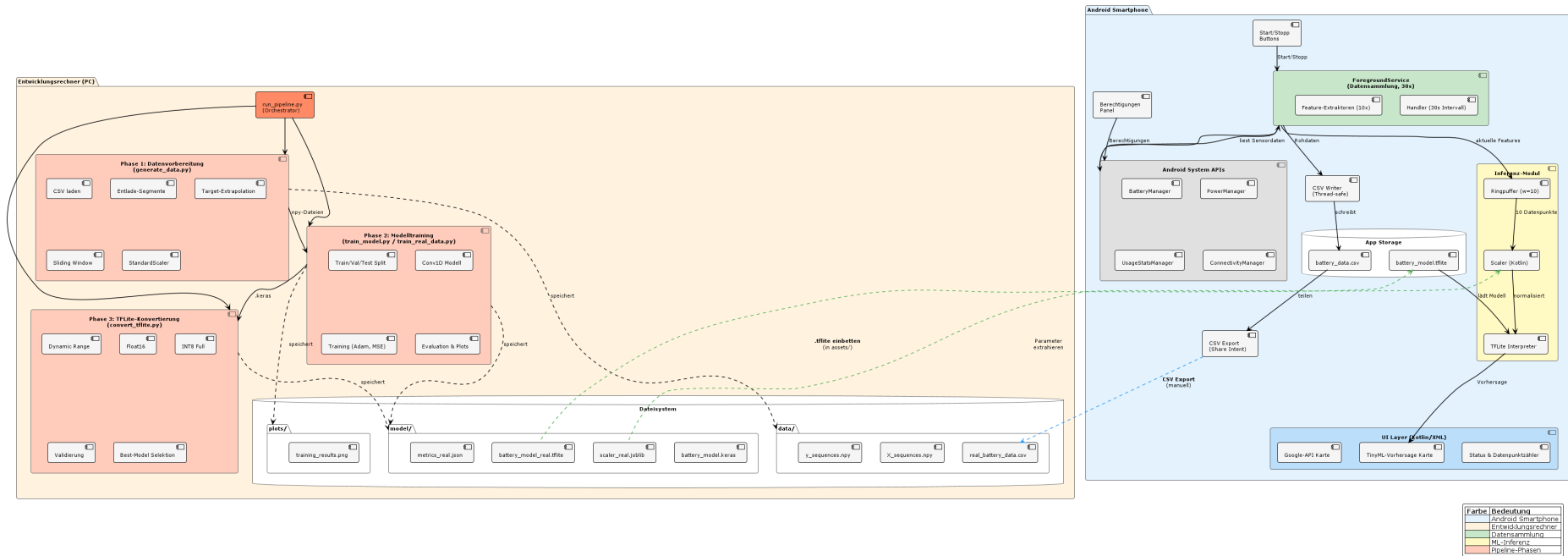


Abbildung 2: Detaillierte Gesamtarchitektur des Battery-Predictor-Systems

1.2 Android-App im Detail

Die App ist in sechs Kotlin-Dateien gegliedert:

Tabelle 2: Module der Android-App

Datei	Aufgabe
MainActivity.kt	UI mit Start/Stop/Export-Buttons, zeigt aktuelle Vorhersage
DataCollectorService.kt	Foreground-Service, sammelt alle 30 Sekunden Daten
BatteryDataPoint.kt	Datenklasse für einen Messpunkt mit CSV-Serialisierung
BatteryDataLogger.kt	Schreibt CSV in filesDir, behandelt Schema-Drift
BatteryPredictor.kt	TFLite-Interpreter mit StandardScaler-Normalisierung
BootReceiver.kt	Startet den Service nach Geräte-Neustart

1.3 Python-Pipeline im Detail

Die Auswerte-Seite ist nach *Forschungs-Methodik* organisiert (eigener Ordner pro Methode, separater Eval-Ordner). Der Orchestrator `run_pipeline.py` bringt sie in Reihenfolge:

1. `methods/tinyml/merge_devices.py` – alle Geräte-CSVs zu einer kombinierten CSV zusammenführen
2. `methods/tinyml/data_prep.py` – Discharge-Segmente finden, Sliding-Windows bilden, Segment-Level-Split
3. `methods/tinyml/train.py` – Conv1D-Modell trainieren
4. `methods/tinyml/tflite_convert.py` – Konvertierung in drei TFLite-Varianten (dynamic, float16, int8)
5. `methods/*/predict.py` – Vorhersagen aller sechs verglichenen Methoden auf dem Test-Set
6. `evaluation/*.py` – diverse Auswertungs-Skripte

2 Datenmodell

2.1 CSV-Schema

Pro Messpunkt schreibt die App eine Zeile mit folgenden Spalten:

Tabelle 3: Felder eines CSV-Eintrags

Spalte	Typ	Bedeutung
session_id	String (8)	Zufalls-ID pro Service-Start
timestamp	long (ms)	Unix-Zeitstempel
datetime	String	YYYY-MM-DD HH:MM:SS
battery_level	Float 0–100	Akku-Stand in Prozent
screen_on	Int 0/1	Bildschirm an?
brightness	Float 0–100	Helligkeit normalisiert
active_app_category	Int 0–5	0=idle, 1=social, 2=video, 3=gaming, 4=browser, 5=productivity
wifi_on	Int 0/1	WiFi-verbunden?
mobile_data_on	Int 0/1	Mobilfunk aktiv?
charging	Int 0/1	Lädt gerade?
cpu_usage	Float 0–100	CPU-Frequenz-Auslastung (Proxy)
temperature	Float (°C)	Akku-Temperatur
hotspot_on	Int 0/1	WLAN-Hotspot aktiv?
system_estimate_min	Float	Google-API-Schätzung in Minuten
system_personalized	Int –1/0/1	Ist Google-Schätzung gerätspezifisch?
own_prediction_h	Float	Eigene TinyML-Vorhersage (Stunden)
linear_baseline_h	Float	Naive Vorhersage aus BatteryManager-Countern

Die ersten drei Spalten sind Metadaten, die zehn nächsten sind die *Features* für das ML-Modell, die letzten vier sind Vergleichswerte zur Auswertung.

2.2 Warum genau diese zehn Features?

Bei der Auswahl gab es eine harte Regel: *ich darf nur lesen, was eine normale App ohne Spezial-Berechtigung lesen kann*. Damit fielen einige naheliegende Kandidaten weg:

- **Genaue Stromstärke pro App** (PowerStats HAL): nur System-Apps.
- **Battery Health** (Verschleißzustand): teilweise gesperrt.
- **Kernel-Fuel-Gauge**: gesperrt.

Was übrig blieb, sind die zehn Features oben. Sie decken drei Gruppen ab: **Akkuzustand**

(level, temperature, charging), **Nutzungsintensität** (screen, brightness, app_category, cpu_usage), **Konnektivität** (wifi, mobile, hotspot).

2.3 Speicherung und Export

Die CSV liegt im app-internen Verzeichnis (`Context.filesDir`). Das ist nur für diese App lesbar – weder andere Apps noch ein angeschlossener PC können die Datei direkt sehen (siehe Datenschutz-Subsection 2.4). Der Export erfolgt explizit per Share-Intent durch den Nutzer.

Über vier Geräte hinweg wurden in 45 Tagen **66.001 Messpunkte** in **180 Sessions** gesammelt.

2.4 Datenschutz

Was wird erhoben

Die App speichert ausschließlich technische Geräte-Metriken (die Spalten oben). Es gibt *keinen* Zugriff auf:

- Standort (keine `ACCESS_*_LOCATION`-Permission),
- Kontakte, Anrufprotokolle, SMS,
- Mikrofon, Kamera,
- Inhalte von Apps (kein Accessibility-Service, kein Screen-Recording),
- personenbezogene Identifikatoren wie IMEI oder Telefonnummer.

Das einzige potenziell sensible Feld ist `active_app_category`, und auch das nur als *Kategorie* (0–5), nicht als konkreter Paketname.

Wo gespeichert wird

Im internen Verzeichnis der App (`Context.filesDir`). Nur diese App selbst hat Zugriff; andere Apps können nichts lesen (Android-Sandbox); bei Deinstallation werden die Daten automatisch mit entfernt; die Datei wird weder in iCloud noch in Google Backup synchronisiert. Ein Export findet ausschließlich über expliziten Nutzerklick (Share-Intent) statt – es gibt keinen automatischen Upload, keinen Server und keine Telemetrie.

Berechtigungen

Tabelle 4: Im Manifest deklarierte Permissions

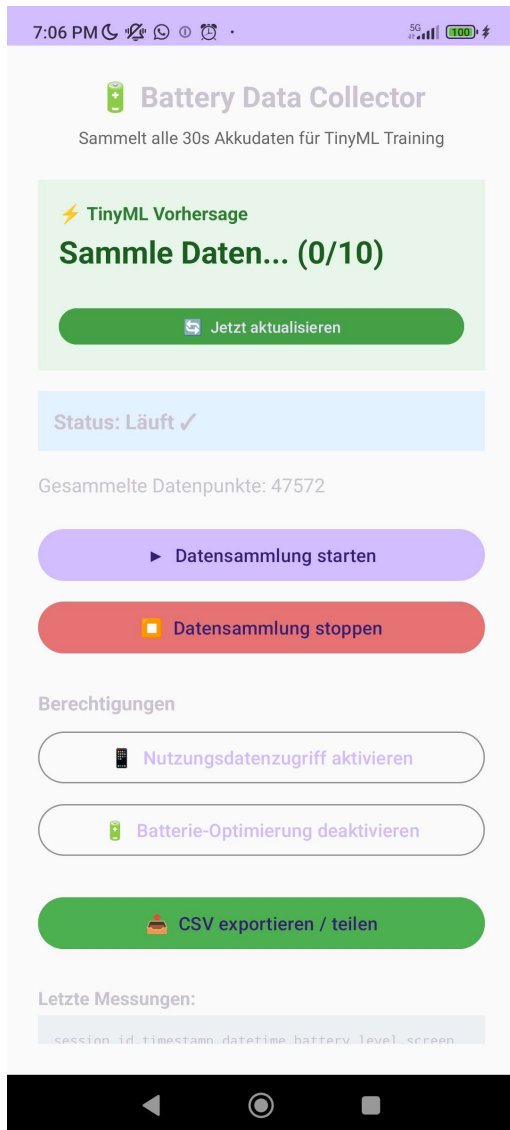
Permission	Wofür
FOREGROUND_SERVICE	Background-Service mit Notification
FOREGROUND_SERVICE_SPECIAL_USE	Service-Typ ab Android 14
WAKE_LOCK	Service läuft auch bei Display aus
RECEIVE_BOOT_COMPLETED	Auto-Restart nach Geräte-Neustart
ACCESS_NETWORK_STATE	WiFi-/Mobile-Daten-Status lesen
ACCESS_WIFI_STATE	WiFi-Status, Hotspot-Reflection
PACKAGE_USAGE_STATS	Aktive App lesen (Nutzer muss manuell aktivieren)
POST_NOTIFICATIONS	Vorhersage in Notification anzeigen
REQUEST_IGNORE_BATTERY_OPTIMIZATIONS	Service vor Doze schützen

Beim Verteilen in der Familie

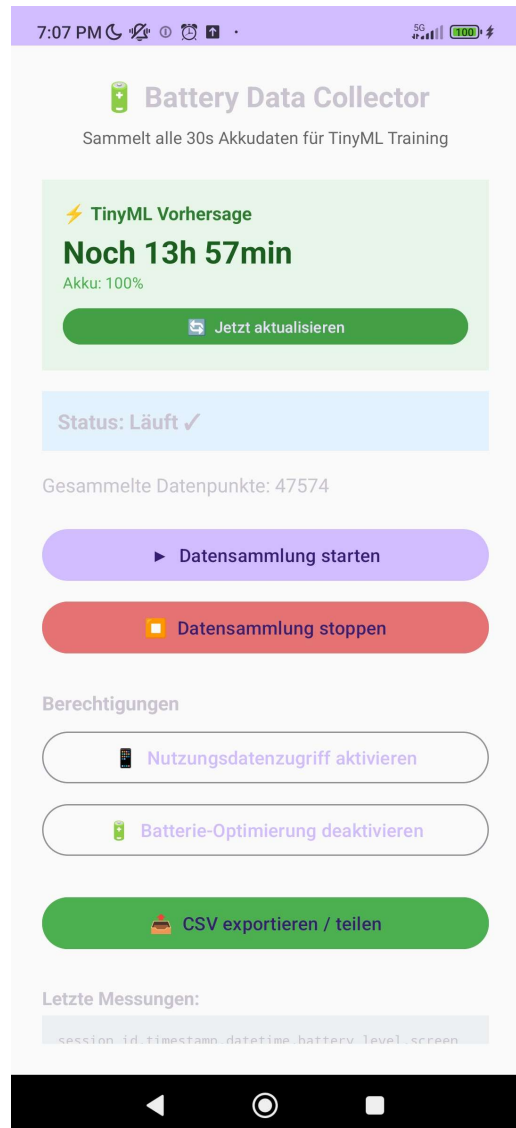
Für die drei Pixel-Geräte habe ich die App-APK an drei Familienmitglieder weitergegeben (per E-Mail bzw. direkt am Phone installiert). Vor der Installation wurde jeweils erklärt, welche Daten gesammelt werden (CSV mit 17 Spalten, davon 10 Sensor-Features plus Metadaten und Vergleichswerte), dass sie nur lokal auf ihrem Phone liegen, dass die CSV nur zu mir kommt, wenn sie sie aktiv per Share-Intent verschicken, und dass keinerlei Auto-Upload oder Hintergrund-Übertragung stattfindet. Eine Einwilligung hat jeder per WhatsApp bestätigt.

2.5 Die App in Aktion

Abbildung 3 zeigt die Main Activity der App auf dem Xiaomi 2107113SG zu zwei Zeitpunkten: einmal direkt nach dem Service-Start (Modell sammelt erst zehn Datenpunkte, also fünf Minuten Aufwärmphase) und einmal mit aktiver TinyML-Vorhersage. Der Datenpunkt-Zähler im unteren Bereich zeigt die Anzahl der vom Service aufgezeichneten Messpunkte (inkl. Lade-Phasen, die später bei der Auswertung herausgefiltert werden) und ist ein wichtiges UX-Element: er gibt dem Nutzer Feedback, dass der Service tatsächlich noch läuft, ohne dass er die Notification öffnen muss.



(a) Aufwärmphase: „Sammle Daten... (0/10)“. Bis zehn Messpunkte vorliegen, gibt das Modell noch keine Vorhersage aus.



(b) Aktive Vorhersage: „Noch 13 h 57 min“ bei 100 % Akku. Die TinyML-Inferenz läuft direkt auf dem Phone.

Abbildung 3: Main Activity der Battery Data Collector App auf dem Xiaomi 2107113SG (Redmi Note 10 Pro). Links: erste Sekunden nach Service-Start. Rechts: dasselbe Gerät eine Minute später mit fertiger On-Device-Vorhersage.

3 Programmierung

3.1 Datensammlung im Service

Der Kern des DataCollectorService ist ein Handler.postDelayed-Loop, der alle 30 Sekunden eine Messung auslöst:

```

1  private val INTERVAL_MS = 30 * 1000L // 30 Sekunden
2
3  private val collectRunnable = object : Runnable {
4      override fun run() {
5          collectAndLog()
6          handler.postDelayed(this, INTERVAL_MS)
7      }
8  }
9
10 private fun collectAndLog() {
11     // 1) Roh-Features lesen
12     val batteryLevel = getBatteryLevel()
13     val screenOn = if (isScreenOn()) 1 else 0
14     val brightness = getScreenBrightness()
15     // ... (10 Features insgesamt)
16
17     // 2) TinyML-Vorhersage (fail-safe)
18     val features = floatArrayOf(batteryLevel, screenOn.toFloat(), /*
19         ... */
20     var prediction = -1f
21     try {
22         predictor?.let { prediction = it.addDataAndPredict(features)
23     } catch (e: Exception) {
24         Log.e("BatteryCollector", "Prediction failed: ${e.message}")
25     }
26
27     // 3) Vergleichswerte einlesen
28     val systemEstimateMin = getSystemBatteryEstimate()
29     val linearBaselineH = getLinearBaselineHours()
30
31     // 4) Datenpunkt schreiben
32     logger.log(BatteryDataPoint(/* ... */), sessionId)

```

Quellcode 1: Mess-Schleife im DataCollectorService

3.2 Auslesen der 10 Features

Die meisten Werte kommen direkt aus den Android-System-Services. Drei Sachen waren tricky.

CPU-Auslastung

Seit Android 8 ist `/proc/stat` für Apps gesperrt. Statt der „echten“ CPU-Auslastung lese ich die aktuelle CPU-Frequenz pro Kern und vergleiche mit der Maximalfrequenz. Höhere Frequenz = höhere Last:

```
1 private fun getAvgCpuFrequencyPercent(): Float {
2     val numCores = Runtime.getRuntime().availableProcessors()
3     var totalRatio = 0f
4     var readCores = 0
5     for (i in 0 until numCores) {
6         try {
7             val cur = File("/sys/devices/system/cpu/cpu$i/cpufreq/
              scaling_cur_freq")
8                 .readText().trim().toLong()
9             val max = File("/sys/devices/system/cpu/cpu$i/cpufreq/
              cpuinfo_max_freq")
10                .readText().trim().toLong()
11            if (max > 0) {
12                totalRatio += cur.toFloat() / max
13                readCores++
14            }
15        } catch (_: Exception) { }
16    }
17    if (readCores == 0) throw Exception("No CPU freq readable")
18    return (totalRatio / readCores * 100f).coerceIn(0f, 100f)
19 }
```

Quellcode 2: CPU-Last als Frequenz-Proxy

Aktive App

Mit dem `UsageStatsManager` (Permission `PACKAGE_USAGE_STATS`, die der Nutzer manuell in den Einstellungen aktivieren muss) lese ich die zuletzt benutzte App und ordne sie über ein Package-Prefix-Mapping einer Kategorie zu:

```
1 private val APP_CATEGORIES = mapOf(
2     "com.instagram"           to 1, // social
3     "com.whatsapp"            to 1,
4     "org.telegram"            to 1,
5     "com.google.android.youtube" to 2, // video
6     "com.netflix"             to 2,
```

```

7      "com.spotify"                to 2,
8      "com.supercell"              to 3, // gaming
9      "com.miHoYo"                 to 3,
10     "com.google.android.apps.chrome" to 4, // browser
11     "org.mozilla"                 to 4,
12     "com.google.android.apps.docs" to 5, // productivity
13     "com.microsoft"               to 5,
14     // ... (rund 30 Eintraege)
15 )

```

Quellcode 3: App-Kategorisierung (Auszug)

Unbekannte Packages bekommen Kategorie 0 (idle), werden aber zusätzlich geloggt, damit ich sie später nachmappen kann.

Hotspot-Status

Die offizielle `WifiManager.isWifiApEnabled()`-API ist als `@hide` markiert, also nicht im SDK. Über Reflection ist sie trotzdem erreichbar:

```

1 private fun isHotspotOn(): Boolean {
2     return try {
3         val wm = applicationContext.getSystemService(Context.
4             WIFI_SERVICE) as WifiManager
5         val method = wm.javaClass.getDeclaredMethod("isWifiApEnabled")
6         method.isAccessible = true
7         method.invoke(wm) as Boolean
8     } catch (e: Exception) {
9         false
10    }
11 }

```

Quellcode 4: Hotspot-Status via Reflection

3.3 On-Device-Inferenz mit TFLite

Der `BatteryPredictor` hält das TFLite-Modell und einen Ringpuffer mit den letzten 10 Datenpunkten. Erst wenn der Puffer voll ist, wird vorhergesagt. Wichtig: die Features müssen mit denselben `StandardScaler`-Werten normalisiert werden, mit denen das Modell trainiert wurde – die Werte sind hardcoded in der App und werden bei jedem Re-Training neu hineingeschrieben.

```

1 class BatteryPredictor(context: Context) {
2     companion object {
3         const val SEQUENCE_LENGTH = 10

```

```

4      const val NUM_FEATURES = 10
5      // Werte aus models/scaler.joblib (Multi-Device-Training)
6      private val SCALER_MEAN = floatArrayOf(
7          68.79f, 0.83f, 8.47f, 1.31f, 0.08f,
8          0.92f, 0.00f, 46.35f, 33.00f, 0.07f
9      )
10     private val SCALER_SCALE = floatArrayOf(
11         29.61f, 0.38f, 13.01f, 1.17f, 0.27f,
12         0.27f, 1.00f, 16.93f, 4.21f, 0.26f
13     )
14 }
15 private val interpreter: Interpreter
16 private val recentData = ArrayDeque<FloatArray>(SEQUENCE_LENGTH)
17
18 fun addDataAndPredict(features: FloatArray): Float {
19     // NaN/Inf-Guard vor StandardScaler
20     for (v in features) if (v.isNaN() || v.isInfinite()) return -1f
21
22     val normalized = FloatArray(NUM_FEATURES) { i ->
23         val s = SCALER_SCALE[i]
24         if (s > 1e-6f) (features[i] - SCALER_MEAN[i]) / s else 0f
25     }
26     if (recentData.size >= SEQUENCE_LENGTH) recentData.
27         removeFirst()
28     recentData.addLast(normalized)
29     if (recentData.size < SEQUENCE_LENGTH) return -1f
30
31     val input = ByteBuffer.allocateDirect(SEQUENCE_LENGTH *
32         NUM_FEATURES * 4)
33         .order(ByteOrder.nativeOrder())
34     for (step in recentData) for (v in step) input.putFloat(v)
35     val output = ByteBuffer.allocateDirect(4).order(ByteOrder.
36         nativeOrder())
37     interpreter.run(input, output)
38     output.rewind()
39     return output.float.coerceAtLeast(0f)
40 }

```

Quellcode 5: Inferenz mit TFLite

3.4 Robustheit – die App muss überleben

Das war der frustrierendste Teil. Android killt Hintergrund-Services aggressiv, gerade auf Xiaomi-Geräten mit MIUI-Aufsatz. Drei Mechanismen sorgen dafür, dass die Datensamm-

lung nicht abreißt:

1. **Foreground-Service** mit dauerhafter Notification.
2. **Auto-Restart nach Wegwischen** der App-Karte. Wenn der Nutzer die App in der Übersicht wegwischt, registriert sich der Service über `AlarmManager.setExactAndAllowWhileIdle` einen Restart in 1 Sekunde.
3. **Auto-Restart nach Geräte-Neustart** über einen `BootReceiver`, der das `BOOT_COMPLETED`-Intent abfängt – aber nur, wenn der Nutzer den Sammler explizit aktiviert hatte.

```

1  override fun onTaskRemoved(rootIntent: Intent?) {
2      val restartIntent = Intent(this, DataCollectorService::class.java
3      )
4      val pendingIntent = PendingIntent.getService(
5          this, 99, restartIntent,
6          PendingIntent.FLAG_ONE_SHOT or PendingIntent.FLAG_IMMUTABLE
7      )
8      val alarm = getSystemService(Context.ALARM_SERVICE) as
9      AlarmManager
10     alarm.setExactAndAllowWhileIdle(
11         AlarmManager.ELAPSED_REALTIME_WAKEUP,
12         SystemClock.elapsedRealtime() + 1000,
13         pendingIntent
14     )
15     super.onTaskRemoved(rootIntent)
16 }

```

Quellcode 6: Auto-Restart bei onTaskRemoved

3.5 Schema-Drift im Logger

Beim Hinzufügen neuer Spalten (etwa `system_personalized` oder `linear_baseline_h`) hatten bestehende CSV-Dateien weniger Spalten als die neue Version erwartete. Der Logger prüft deshalb beim Start die Header-Zeile und parkt alte CSVs automatisch:

```

1  init {
2      val expectedHeader = "session_id,timestamp,datetime,{
3      BatteryDataPoint.CSV_HEADER}"
4      if (!csvFile.exists() || csvFile.length() == 0L) {
5          csvFile.writeText("$expectedHeader\n")
6      } else {
7          val firstLine = csvFile.useLines { it.firstOrNull() ?: "" }
8          if (firstLine != expectedHeader) {
9              val backup = File(context.filesDir,
10                  "battery_data_legacy_${System.currentTimeMillis()}.
11                  csv")

```

```

10         csvFile.copyTo(backup, overwrite = true)
11         csvFile.writeText("$expectedHeader\n")
12     }
13 }
14 }

```

Quellcode 7: Schema-Drift im Logger abfangen

3.6 Python-Pipeline – Beispiel Segment-Erkennung

Die wichtigste Pipeline-Funktion findet Entlade-*Segmente*: zusammenhängende Phasen, in denen das Phone entladen wird, ohne zwischendurch zu laden. Daraus werden später Sliding-Window-Sequenzen gebildet.

```

1 def find_discharge_segments(df, min_length, max_gap_ms):
2     segments = []
3     for _, session_df in df.groupby("session_id"):
4         session_df = session_df.sort_values("timestamp").reset_index(
5             drop=True)
6         is_discharging = session_df["charging"] == 0
7         block_id = (~is_discharging).cumsum() # neuer Block bei
8             Laden
9         for _, block in session_df[is_discharging].groupby(block_id[
10             is_discharging]):
11             # innerhalb des Blocks bei groesseren Zeitluecken
12             splitten
13             time_diffs = block["timestamp"].diff()
14             gap_id = (time_diffs > max_gap_ms).cumsum()
15             for _, sub in block.groupby(gap_id):
16                 if len(sub) >= min_length:
17                     segments.append(sub.reset_index(drop=True).copy())
18             )
19     return segments

```

Quellcode 8: Discharge-Segmente finden

3.7 Trainings-Verlauf

Abbildung 4 zeigt den Trainings- und Validierungs-Verlauf des Conv1D-Modells über die 21 Epochen, nach denen EarlyStopping greift.

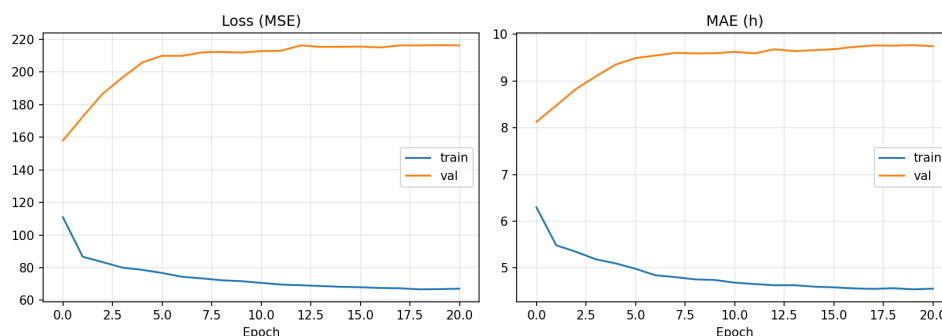


Abbildung 4: Trainings- und Validation-MAE des Conv1D-Modells. Auf dem Multi-Device-Datensatz erreicht das Modell nach 21 Epochen ein Validation-MAE von ca. 5 h. Da EarlyStopping mit `restore_best_weights=True` konfiguriert ist, wird das beste Modell zurückgegeben.

3.8 Code-Umfang und Eigenleistung

Über die sieben Bearbeitungswochen sind drei Schichten parallel entstanden – die Android-App in Kotlin, die ML-Pipeline am PC und die Auswertungs-Schicht. Tabelle 5 zeigt den Umfang nach Bereich; die wichtigsten Brocken sind die Auswertungs-Schicht (≈ 2.050 Zeilen reine Auswertungs-Logik) und die Android-App (≈ 850 Zeilen Kotlin, die produktionsnah auf vier Geräten laufen).

Tabelle 5: Code-Umfang nach Bereich

Bereich	Zeilen (ohne Leerzeilen)
Android-App (6 Kotlin-Dateien)	≈ 850
Python-Pipeline (<code>methods/</code> , 13 Module)	≈ 1.150
Python-Auswertung (<code>evaluation/</code> , 12 Module)	≈ 2.050
Konfiguration und Orchestrator	≈ 300
Summe	≈ 4.350

Selbst geschrieben:

- Gesamte Android-App-Struktur (Activity, Service, BroadcastReceiver, FileProvider), inklusive `onTaskRemoved`-Auto-Restart und Schema-Drift-Detection.
- Auswahl und Implementierung der zehn Features (CPU-Frequenz-Proxy, App-Kategorie-Mapping, Hotspot-Reflection).
- Datenmodell, CSV-Schema und FileProvider-Konfiguration für den Share-Export.

- Pipeline-Architektur (sechs verglichene Methoden je eigenes Modul, Trennung von Methoden- und Auswertungs-Schicht).
- Konkrete Implementierung der analytischen Baselines (Linear, Exponential mit Fall-back und Cap gegen Divergenz).
- Segment-Level-Split als Anti-Leakage-Mechanismus.
- Auswertungs-Skripte: Genauigkeits-Metriken, Per-Device-Analyse, Daten-Diagnose.

Übernommen:

- Conv1D-Architektur in Anlehnung an gängige TinyML-Topologien [?, ?] (zwei Convolutions, GlobalAveragePooling, zwei Dense).
- TFLite-Konvertierungs-API (Standard-Funktionen von TensorFlow).
- `scikit-learn`: `StandardScaler`, `RandomForestRegressor` [?], `train_test_split`.
- `scipy`: `curve_fit` für den exponentiellen Fit.

Probleme und wie sie gelöst wurden:

- *Service stirbt auf MIUI*: Auto-Restart über AlarmManager.
- */proc/stat gesperrt*: Frequenz-Proxy statt echter Auslastung.
- *CSV-Schema-Migration*: Header-Check beim Service-Start.
- *TinyML auf Single-Device zeigt MAE 0.5h Artefakt*: Segment-Level-Split eingeführt, korrigiert den scheinbaren Wert auf 9.96 h.
- *Exponential-Fit divergiert manchmal in 30.000h*: 72-h-Cap als Sanity-Bound.

3.9 Reproduzierbarkeit der Pipeline

Damit die Auswertung nicht von Hand-Schritten abhängt, gibt es einen einzigen Orchestrator `run_pipeline.py`, der die komplette Verarbeitungskette ausführt – vom Mergen der Geräte-CSVs bis zu den fertigen Auswertungs-Reports.

```
1 # vollstaendige Pipeline: ca. 20 Minuten auf Notebook-CPU
2 python run_pipeline.py
3
4 # nur die ML-Schritte (ohne Eval) -- ca. 5 Minuten
5 python run_pipeline.py --stage tinyml
```

Quellcode 9: Pipeline reproduzierbar starten

Alle Hyperparameter, Feature-Listen, Random-Seeds und Pfade liegen in `configs/default.yaml`. Beim Re-Training werden zusätzlich die `StandardScaler`-Werte für die Android-App neu exportiert (`methods/tinyml/export_scaler_for_android.py`) und ein Check-Script vergleicht sie mit den hardcoded Werten in `BatteryPredictor.kt`, damit die Normalisierung in App und Modell garantiert übereinstimmt.

4 Kurzreferat (Folien)

Für den 5-minütigen Pitch zum Portfolio habe ich eine eigene 8-Folien-Kurzpräsentation unter `portfolio/App_Kurzpraesi.pptx` erstellt. Sie ist ausschließlich auf die App ausgerichtet; der wissenschaftliche Methoden-Vergleich (Paper-Ergebnisse) bleibt bewusst außen vor.

1. Titel (Battery Data Collector, Vertiefung I, 5 Minuten).
2. Was die App macht: Sammeln – Rechnen – Anzeigen.
3. Die App in Aktion: Screenshots (Aufwärmphase und aktive Vorhersage auf dem Xiaomi).
4. **Hauptfeature, Teil 1:** On-Device-Inferenz – was „on-device“ konkret heißt und warum es ein gutes Feature ist (Datenschutz, Offline, Latenz, keine Server).
5. **Hauptfeature, Teil 2:** wie die Inferenz technisch abläuft (5-Schritt-Pipeline) und Modell-Steckbrief (Conv1D, TFLite INT8, 14.4 KB, $\sim 5 \mu s$).
6. Das größte App-Problem: MIUI killt Hintergrund-Services – Foreground-Service + AlarmManager + BootReceiver als Lösung.
7. Datenschutz (keine Standort/Mikro/Kamera, alles lokal) und Stand: 4 Geräte, 45 Tage, 66.001 Messpunkte.
8. Fragen + Kontakt.

Der Schwerpunkt liegt auf dem *Hauptfeature* (Folien 4 und 5 zusammen rund zwei Minuten). Folien 2–3 und 6–7 liefern den App-Kontext drumherum, Folie 1 und 8 sind Rahmen.

5 Abschließender Erfahrungsbericht

5.1 Wöchentliche Einträge

Woche 1 (KW 13: 23.–29. März)

Setup und erste Datensammlung. Repo angelegt, Android-Projekt aufgesetzt, erste Version des `DataCollectorService` geschrieben. Schon am 25. März lief die erste Messung auf dem Xiaomi durch. Hauptproblem in dieser Woche: die `PACKAGE_USAGE_STATS`-Permission ist nicht via Standard-Dialog vergebbar – der Nutzer muss in den Einstellungen manuell ankreuzen. Ich habe deshalb einen Button in der App eingebaut, der direkt zum `ACTION_USAGE_ACCESS_SETTINGS` springt.

Woche 2 (KW 14: 30. März – 5. April)

Service-Robustheit. Datensammlung bricht beim Wegwischen aus der App-Übersicht ab. Drei Tage damit verbracht herauszufinden, wie man auf Xiaomi-MIUI einen Service vor dem Aggressive-Killing schützt. Lösung: `onTaskRemoved` → `AlarmManager.setExactAndAllowWhileIdle` → Service kommt 1 Sekunde später wieder. Plus `BootReceiver` damit nach Neustart automatisch weitergesammelt wird.

Woche 3 (KW 15: 6.–12. April)

Erstes Modell, erste Auswertung – erstes Erschrecken. Genug Daten zusammen (~11.000 Messpunkte), erstes Training durchgelaufen, MAE 0.53 h. Habe das stolz aufgeschrieben. Dann stutzig geworden: warum ist das so viel besser als die Google-API? Beim Nachprüfen festgestellt, dass das ein klassisches Data-Leakage-Artefakt vom Random-Split war. Pipeline auf Segment-Level-Split umgestellt → MAE wird zu 9.96 h. Das war der methodische Wendepunkt der Arbeit. Am 12. April App-APK an ein Familienmitglied mit Pixel 7 Pro weitergegeben.

Woche 4 (KW 16: 13.–19. April)

Multi-Device wird real. APK an zwei weitere Familienmitglieder verteilt (Pixel 8 Pro, Pixel 9 Pro XL). Wichtig vor allem die Aufklärung darüber, was genau gesammelt wird – ich habe einen Mini-Datenschutz-Hinweis schriftlich geschickt und mündlich erklärt. Alle drei haben zugestimmt. Parallel das CSV-Schema erweitert (`system_personalized`, `linear_baseline_h`) und den Logger so gebaut, dass er bei Schema-Wechsel die alte Datei sicher parkt.

Woche 5 (KW 17: 20.–26. April)

Auswertungs-Tiefe. Pipeline so umgebaut, dass sie alle vier Geräte-CSVs konsolidiert und auswertet. Mean-Predictor und Random Forest als Sanity-Modelle hinzugefügt, weil ein „TinyML schlägt nichts“ wenig aussagekräftig wäre, ohne zu zeigen womit es überhaupt verglichen wird. Dazu noch die analytischen Baselines (Linear, Exponential) implementiert.

Woche 6 (KW 18: 27. April – 3. Mai)

Auswertungs-Infrastruktur fertig stellen. Auswertungs-Skripte für Modell-Genauigkeit, Gerät-Vergleiche und Datenverteilungs-Diagnose in die Pipeline integriert. Zwischendurch die Xiaomi-Datensammlung beendet (28. April) – die finale Datenmenge stand fest: 66.001 Messpunkte. Der Wechsel von Single-Device auf Multi-Device zeigte spürbar klarere Unterschiede zwischen den Methoden als noch zuvor; das hat die Entscheidung bestätigt, die Datensammlung über mehrere Geräte hinweg auszudehnen.

Woche 7 (KW 19/20: 4.–11. Mai)

Konsolidierung und Dokumentation. Pixel-Datensammlung beendet (10. Mai). Die Per-Device-Auswertung zeigte deutlich unterschiedliche Modell-Performance je nach Gerät; beim Hineinschauen in die Datenverteilung wurde der Grund sichtbar: auf dem Xiaomi war der Akku 88 % der Zeit über 75 %, also kaum Entlade-Dynamik zum Lernen vorhanden. Diese Woche viel Zeit in die Aufbereitung beider Abgabe-Dokumente (Portfolio und separates Paper) gesteckt, in Pipeline-Konsistenz-Checks und in das Schreiben dieses Erfahrungsberichts.

5.2 Gesamt-Aufwand

Tabelle 6: Zeitaufwand nach Woche

Woche	Hauptaufwand	Stunden
1	App-Grundgerüst + Permissions	~ 15
2	Service-Robustheit auf MIUI	~ 18
3	Trainings-Pipeline + Leakage-Aha-Moment	~ 20
4	Schema-Migration + APK-Verteilung	~ 10
5	6-Wege-Vergleich + analytische Baselines	~ 18
6	Auswertungs-Skripte + Daten-Diagnose	~ 15
7	Portfolio/Paper schreiben + Audits	~ 25
Summe		≈ 120 h

5.3 KI-Nutzung

Während des Projekts habe ich **Claude Code** (Anthropic, Modell Opus 4.7 mit 1M-Token-Kontext) als Programmier-Assistent verwendet. Dieser Abschnitt ist bewusst transparent gehalten.

Wofür Claude genutzt wurde:

- *Code-Refactoring* – Umstrukturierung von einem monolithischen Trainings-Skript zu einer modularen Pipeline (`methods/`, `evaluation/`).
- *Boilerplate* – TFLite-Konvertierungs-Boilerplate, Argument-Parser, Standard-Plot-Code (matplotlib).
- *Recherche-Beschleunigung* – Suche nach relevanter Android- und TinyML-Literatur.
- *Auswertungs-Skripte* – Modell-Genauigkeit, Gerät-spezifische Auswertung und Daten-Diagnose nach meinen Vorgaben.
- *Schreibhilfe* – Strukturierung dieses Portfolios und des separaten Papers, Tabellen-Formatierung, Korrektur-Lesen.
- *Audit-Runden* – systematisches Aufspüren eigener Fehler (inkonsistente Zahlen, methodische Schwächen, Confounder-Verdacht).

Was ich selbst gemacht habe:

- Das gesamte Projekt-Konzept und die App-Idee.
- Auswahl der zehn Features (was eine normale App ohne Spezial-Rechte überhaupt lesen darf).
- Datensammlung auf den vier Geräten – inklusive App-Verteilung an drei Familienmitglieder.
- Alle methodischen Entscheidungen: Feature-Auswahl, Segment-Level-Split (Trainingsdaten-Aufbereitung), Auswahl der drei in der App angezeigten Vergleichswerte.
- Kritisches Hinterfragen der KI-Vorschläge, wenn etwas verdächtig wirkte. Im Verlauf wurden so mehrere eigene Fehler (z. B. falsche Autoren bei drei Quellen, zu plakative Aussagen) korrigiert.
- Alle Anpassungen am Android-Manifest, Permissions, geräte-spezifisches Debugging (MIUI-Service-Stirbt-Problem).

Bilanz. Realistisch eingeschätzt: ohne KI-Unterstützung hätte ich das Projekt in dieser Tiefe nicht in sieben Wochen geschafft. Die KI war besonders nützlich für Boilerplate

und Recherche-Beschleunigung sowie für methodische Disziplin (Konsistenz-Checks über große Code-Mengen). Was die KI *nicht* ersetzt hat: die Entscheidung, was gemacht wird, das Verstehen der Ergebnisse, das Verteidigen von Trade-offs, und vor allem die handfeste Geräte-Logistik und das Debugging vor Ort.

5.4 Abschluss-Reflexion

Was funktioniert (Stand 11. Mai 2026):

- Android-App läuft stabil im Hintergrund auf vier verschiedenen Geräten über mehrere Wochen.
- TFLite-Modell ist 14.4 KB groß, läuft auf der CPU des Phones in $\sim 5 \mu\text{s}$ pro Inferenz.
- Datensammlung umfasst 66.001 Messpunkte, sauber per Share-Intent exportiert.
- Trainings- und Auswertungs-Pipeline reproduzierbar via einem Befehl (`python run_pipeline.py`)

Was noch nicht ideal ist:

- Die App-UI ist zweckmäßig, aber nicht hübsch. Material-3 ist nur das Default-Theme.
- Die Scaler-Werte werden per Skript aus dem Python-Modell ins Kotlin kopiert – ein automatischer Build-Step wäre eleganter.
- Es gibt kein In-App-Onboarding für die UsageStats-Permission, nur einen Button-Hinweis.

Was ich aus dem Projekt mitnehme:

- *Android im Hintergrund ist hart.* Theoretisch sind Foreground-Services eine Standard-Lösung. Praktisch kilt jeder Hersteller anders. Multi-Device-Test ist die einzige Möglichkeit, das wirklich zu prüfen.
- *Methodische Disziplin schlägt Modell-Komplexität.* Der Wechsel von Random-Split zu Segment-Level-Split hatte mehr Einfluss auf das Ergebnis als jedes Hyperparameter-Tuning.
- *Negative Ergebnisse ehrlich darstellen ist genau so wertvoll wie positive.* Anfangs schien das Projekt zu „funktionieren“ (MAE 0.5 h). Erst durch saubere Methodik kam heraus, dass dieser Wert ein Leakage-Artefakt war. Das ehrlich aufzuschreiben ist anstrengender, aber wissenschaftlich korrekter.

Repository: <https://github.com/Keno-fsdf/HandyUsageRepo>

Eingereicht am *12. Mai 2026* – Keno Schürger